

PostgreSQL 17 Logical Replication — Step-by-Step

Overview

Logical replication streams individual data changes (INSERT, UPDATE, DELETE) from one database (publisher) to another (subscriber) on the same PostgreSQL instance. Unlike physical/streaming replication, logical replication works at the table level and allows selective replication.

Setup used:

- Single EC2 instance (Amazon Linux 2023)
 - PostgreSQL 17
 - Publisher: `publisher_db`
 - Subscriber: `subscriber_db`
 - Replicated table: `orders`
-

Step 1 — Enable Logical Replication

PostgreSQL's WAL (Write-Ahead Log) level must be set to `logical`.

```
sudo -u postgres psql -c "ALTER SYSTEM SET wal_level = logical;"
sudo systemctl restart postgresql
```

Verify:

```
sudo -u postgres psql -c "SHOW wal_level;"
# Expected: logical
```

Step 2 — Fix `pg_hba.conf` Authentication

By default, Amazon Linux 2023 PostgreSQL uses `ident` auth for local connections, which blocks password-based replication connections. Change all host entries to `md5`.

```
sudo sed -i 's/ident/md5/g; s/scram-sha-256/md5/g' /var/lib/pgsql/data/pg_hba.conf
sudo systemctl reload postgresql
```

Verify:

```
sudo grep "^host" /var/lib/pgsql/data/pg_hba.conf
# All entries should show md5
```

Step 3 — Set a Password for the `postgres` User

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'Admin1234!';"
```

Step 4 — Create the Publisher Database and Table

```
sudo -u postgres psql -c "CREATE DATABASE publisher_db;"
```

```
sudo -u postgres psql -d publisher_db -c "  
CREATE TABLE orders (  
  id          SERIAL PRIMARY KEY,  
  customer    VARCHAR(100),  
  product     VARCHAR(100),  
  amount      NUMERIC(10,2),  
  created_at  TIMESTAMP DEFAULT now()  
);"
```

Step 5 — Create the Publication

A publication defines which tables and operations are replicated.

```
sudo -u postgres psql -d publisher_db -c "CREATE PUBLICATION pub_orders FOR TABLE orders;"
```

Verify:

```
sudo -u postgres psql -d publisher_db -c "\dRp"
```

Step 6 — Create the Subscriber Database and Table

The subscriber must have the same table structure. The sequence default can differ but the columns must match.

```
sudo -u postgres psql -c "CREATE DATABASE subscriber_db;"
```

```
sudo -u postgres psql -d subscriber_db -c "  
CREATE TABLE orders (  
  id          SERIAL PRIMARY KEY,  
  customer    VARCHAR(100),  
  product     VARCHAR(100),  
  amount      NUMERIC(10,2),  
  created_at  TIMESTAMP DEFAULT now()  
);"
```

Step 7 — Create the Replication Slot Manually on the Publisher

`CREATE SUBSCRIPTION` can hang when trying to create the slot automatically (known issue with loopback connections). Creating the slot manually first avoids this.

```
sudo -u postgres psql -d publisher_db -c "SELECT  
pg_create_logical_replication_slot('sub_orders', 'pgoutput');"
```

Step 8 — Create the Subscription

Point `subscriber_db` at the publisher, referencing the pre-created slot.

```
set +H # Disable bash history expansion (required when password contains !)  
  
sudo -u postgres psql -d subscriber_db -c "CREATE SUBSCRIPTION sub_orders CONNECTION  
'host=127.0.0.1 port=5432 dbname=publisher_db user=postgres password=Admin1234!' PUBLICATION  
pub_orders WITH (create_slot = false, slot_name = 'sub_orders', copy_data = false);"
```

Options explained:

Option	Value	Reason
<code>create_slot</code>	<code>false</code>	Slot was already created manually in Step 7
<code>slot_name</code>	<code>sub_orders</code>	References the slot created in Step 7
<code>copy_data</code>	<code>false</code>	Skip copying pre-existing rows; only replicate new changes

Verify the subscription is enabled:

```
sudo -u postgres psql -d subscriber_db -c "SELECT subname, subenabled, subslotname FROM  
pg_subscription;"
```

Verify the slot is active:

```
sudo -u postgres psql -c "SELECT slot_name, active, active_pid FROM pg_replication_slots;"  
# active should be: t
```

Step 9 — Test Replication

Insert on publisher:

```
sudo -u postgres psql -d publisher_db -c "INSERT INTO orders (customer, product, amount)  
VALUES ('Bob', 'Phone', 499.99);"
```

Check subscriber:

```
sudo -u postgres psql -d subscriber_db -c "SELECT * FROM orders;"  
# Row should appear
```

Update on publisher:

```
sudo -u postgres psql -d publisher_db -c "UPDATE orders SET amount = 599.99 WHERE customer =  
'Bob';"
```

Delete on publisher:

```
sudo -u postgres psql -d publisher_db -c "DELETE FROM orders WHERE customer = 'Bob';"
```

Check subscriber after each operation — changes propagate in near real time.

Key Concepts

Publication — defined on the publisher. Specifies which tables and DML operations to replicate.

Subscription — defined on the subscriber. Connects to a publication and applies changes locally.

Replication slot — tracks how far the subscriber has consumed the WAL stream. Prevents the publisher from discarding WAL that hasn't been sent yet. Must be cleaned up if the subscription is dropped.

WAL level = logical — required on the publisher. Enables logical decoding of the WAL stream.

Troubleshooting

Symptom	Cause	Fix
CREATE SUBSCRIPTION hangs	Slot creation deadlock on loopback	Create slot manually (Step 7)
ident authentication failed	pg_hba.conf uses ident	Change to md5 (Step 2)
!': event not found	Bash history expansion on !	Run set +H before the command
Slot missing after DROP SUBSCRIPTION	DROP SUBSCRIPTION deletes the slot	Recreate slot with pg_create_logical_replication_slot
No rows after subscription created	copy_data = false skips existing rows	Insert new rows to test

Cleanup

```
# On subscriber
sudo -u postgres psql -d subscriber_db -c "DROP SUBSCRIPTION sub_orders;"

# On publisher (slot is dropped automatically by DROP SUBSCRIPTION, but if orphaned)
sudo -u postgres psql -d publisher_db -c "SELECT pg_drop_replication_slot('sub_orders');"
```